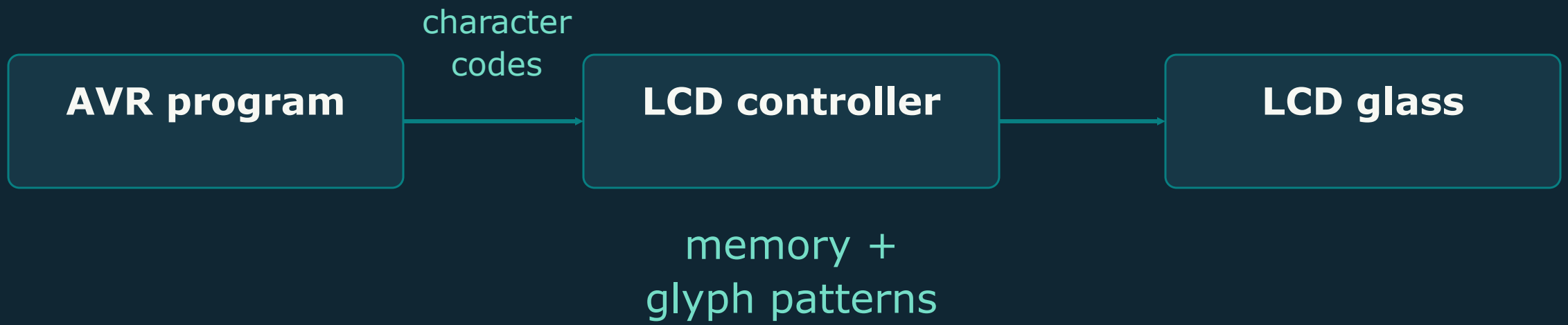
Interfacing AVR with LCD

Arduino Uno and LCD Keypad Shield

Bare-metal AVR C



The LCD keeps displaying while AVR works



The AVR writes a character code.

The LCD controller stores it and drives the glass.

The AVR updates the LCD when the content changes.

Power, light and contrast

VSS connects to GND. VDD supplies the LCD.

The backlight illuminates the glass.

V0 sets character contrast through a potentiometer.

A glowing backlight does not prove initialization.

The shield connections

LCD function	Uno pin	AVR pin
RS	D8	PB0
E	D9	PB1
DB4, DB5	D4, D5	PD4, PD5
DB6, DB7	D6, D7	PD6, PD7
Five keys	A0	PC0 / ADC0
Backlight control	D10	PB2

Hardware readiness

Shield aligned with the Uno headers

R/W tied to GND in the reference design

A0 reserved for keys

A4/A5 free for the sensor

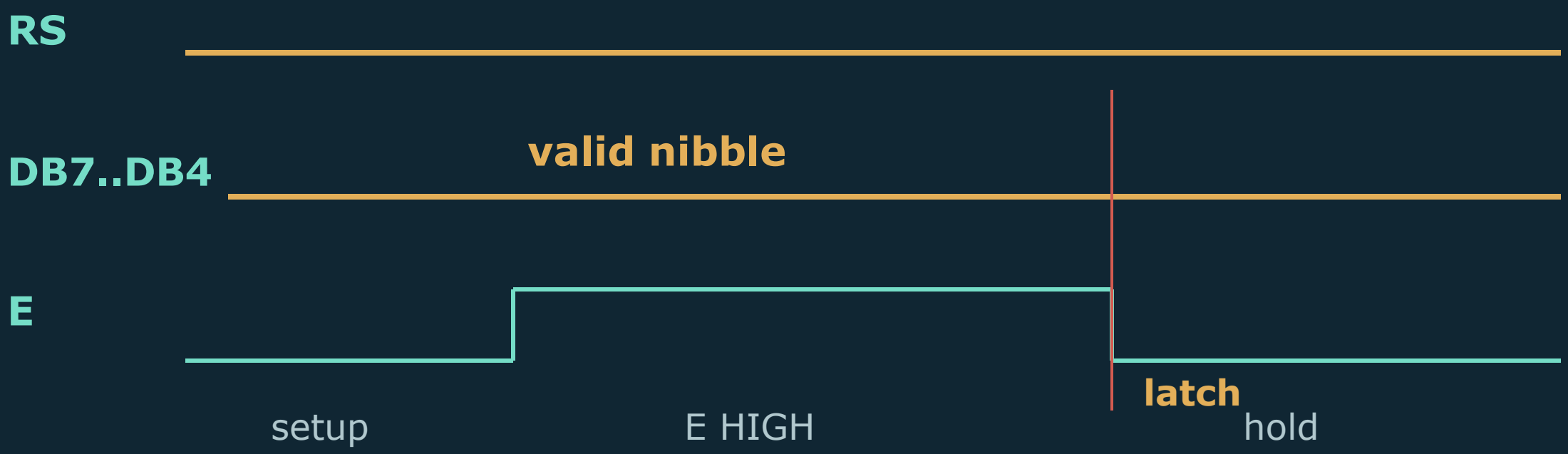
D0/D1 free for UART



One bus, two meanings

RS	R/W	Interpretation
0	0	Command write
1	0	Character or glyph byte write
0	1	Status/address read
1	1	Data read

Enable write timing



Set RS and the data first. Pulse E HIGH, then LOW. Keep data stable around the falling edge.

A byte crosses four data wires twice

0x41

Strobe	DB7	DB6	DB5	DB4
First	0	1	0	0
Second	0	0	0	1

high nibble first, low nibble second

Exercise E1: reconstruct three bytes

After initialization, decode each row. State command/data and the visible result.

RS	First nibble	Second nibble
0	1100	0000
1	0100	0001
1	0100	0010

Solution E1: reconstruct three bytes

0xC0 with RS=0 selects row 2, column 1.

0x41 with RS=1 writes A.

0x42 with RS=1 writes B.

Row 2 begins with AB.

Memory inside the LCD

LCD component	What it holds
DDRAM, 80 bytes	Character codes and their display positions
CGROM	Factory glyph patterns
CGRAM, 64 bytes	Eight custom 5×8 glyphs
Address counter	Current DDRAM or CGRAM location

The two visible rows have different offsets

Row 1

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F

Row 2

40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F

Command = 0x80 | DDRAM address

Exercise E2: place a label

Place X at row 2, column 6.

Give the DDRAM address, command byte and C call.

Solution E2: place a label

Human row 2, column 6 means C row 1, column 5.

DDRAM address = 0x40 + 5 = 0x45.

Command = 0x80 | 0x45 = 0xC5.

lcd_cursor(1, 5); lcd_data('X');

Essential command bytes

Byte	Meaning
0x28	4-bit, two lines, 5×8 font
0x0C	Display on, cursor and blink off
0x06	Increment address, no display shift
0x01 / 0x02	Clear display / return home
0x80 | address	Select a DDRAM address

Two kinds of waiting

Wait	Purpose	Classroom driver
E setup / high / hold	The LCD accepts a nibble	1 µs / 1 µs / 1 µs
Ordinary byte completion	LCD finishes the instruction	100 µs
Clear or home completion	Long internal operation finishes	2 ms

Four-bit startup synchronization

Wait 50 ms before the first strobe.

3	3	3	2
then wait 5 ms	then wait 150 µs	then wait 150 µs	then wait 150 µs

Then bytes: 28, 08, 01, 06, 0C

The first four transfers are individual nibbles.

Masked GPIO preserves other signals

```
PORTD = (uint8_t)((PORTD & 0x0F) |
                  ((value & 0x0F) << 4));
```

PD4..PD7 carry the LCD nibble.

PD0..PD3 retain their previous latch states.

Exercise E3: a register is shared

PORTD currently holds 0x05. Send nibble 0xA.

Calculate the correct new PORTD.

Explain the fault in PORTD = value << 4.

Solution E3: a register is shared

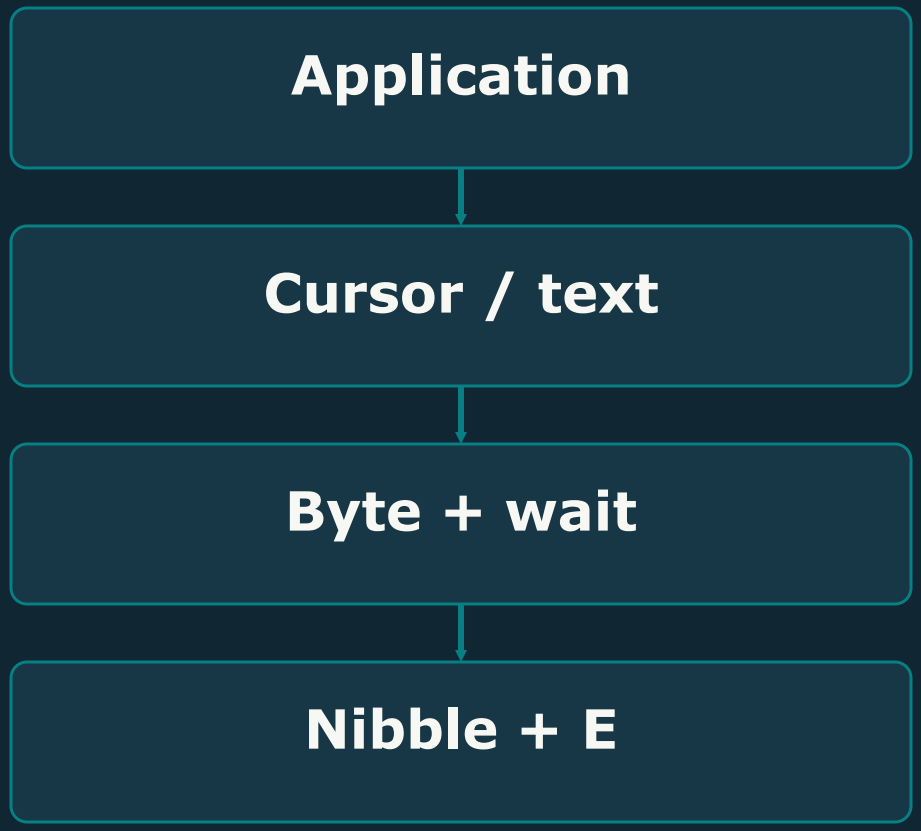
Correct result: 0xA5.

0x05 preserves the lower four latch bits.

0xA0 places the LCD nibble in bits 7..4.

A whole-register write produces 0xA0 and loses the lower bits.

Driver layers



main() handles events and menu data.

lcd_cursor(), lcd_line() position and repaint text.

byte_write() selects RS, sends two nibbles and waits.

nibble() places four bits and pulses E.

A character value and a string pointer

```
void lcd_puts(const char *text)
{
    while (*text)
        lcd_data((uint8_t)*text++);
}

lcd_data('A');
lcd_puts("HELLO");
```

text points to the current character.

*text reads that character value.

text++ advances to the next character.

Bare-metal PlatformIO project

```
[env:uno]
platform = atmelavr@5.1.0
board = uno
board_build.f_cpu = 16000000L
build_flags =
    -std=gnu11
    -Wall
    -Wextra
    -Werror
```

Source file: src/main.c

No framework entry

Dependencies: avr-gcc and avr-libc

Demonstration: the complete display



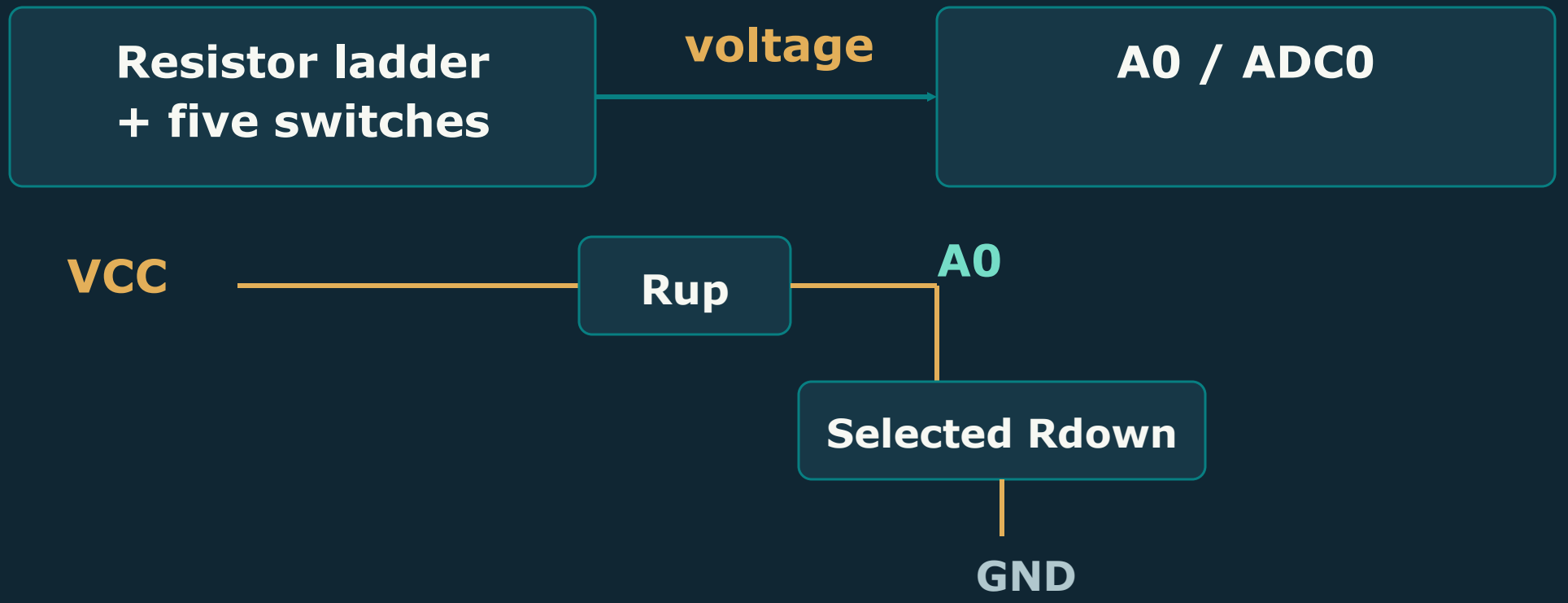
Project 01_LCD_Counter_Glyphs

SELECT increments COUNT once per press.

Holding the key does not repeatedly increment.

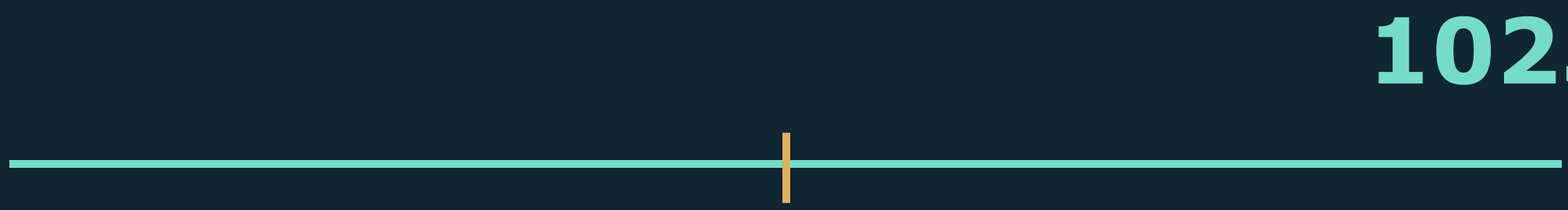
Reset returns COUNT to zero.

Five keys share one analog input



$$V_{out} = VCC \times R_{down} / (R_{up} + R_{down})$$

The ADC measures a fraction of a reference



VA0 ≈ VREF / 2

A0 voltage becomes a number, not an ON/OFF input bit.

A minimal polling ADC0 setup

```
ADMUX = (1 << REFS0);  // AVCC, ADC0
ADCSRA = (1 << ADEN) |
         (1 << ADPS2) |
         (1 << ADPS1) |
         (1 << ADPS0);

ADCSRA |= (1 << ADSC);
while (ADCSRA & (1 << ADSC)) { }
uint16_t raw = ADC;
```

16 MHz / 128 = 125 kHz ADC clock.

Start a conversion and wait for completion.

Read the 10-bit result.

Actual shield calibration

Key	Illustrative center	Starter cutoff
RIGHT	0	<50
UP	144	<195
DOWN	329	<380
LEFT	504	<555
SELECT	741	<790
NONE	near 1023	≥790

Exercise E4: classify a measured key

Use the starter cutoffs on the previous slide.

ADC0 reads 342. Which key is it?

A held UP key ranges from 208 to 216. What must change?

Solution E4: classify a measured key

342 is below 380 but above 195, so it maps to DOWN.

208..216 maps to DOWN under the starter cutoffs.

Measure UP and DOWN ranges on this shield.

Move their boundary into the measured gap and rebuild.

A button level becomes an event

Poll	1	2	3	4	5	6
Candidate	UP	NONE	UP	UP	UP	UP
Action	wait	reset	wait	wait	press	hold

Three matching samples establish the event.

A new key requires a stable release.

Sequential gestures and button limits

A sequence works: SELECT, release, RIGHT.

A long press works after stable samples.

A simultaneous chord gives only one A0 voltage.

Use single-key gestures with explicit release.

A configuration variable in SRAM

LIMIT SETTING

Value: 300

Project 03_SRAM_Menu

LEFT/RIGHT chooses the page.

UP/DOWN changes the value from 100 to 1000.

On MODE, SELECT toggles AUTO/MANUAL.

Menu state and application data



page says what the interface displays.

setting holds the configuration value.

automatic holds the selected mode.

An event changes data, then the display reflects it.

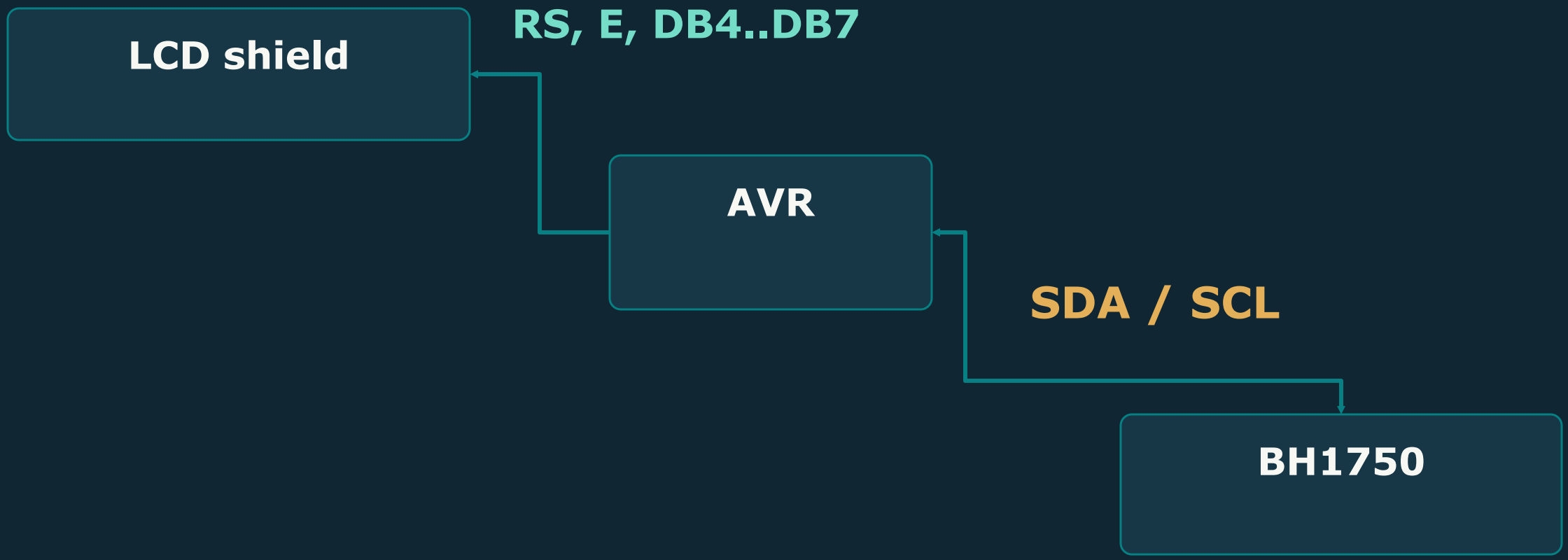
Common LCD symptoms

Observation	First useful checks
Backlight on, blank text	Contrast, init, RS/E mapping
Blocks with no text	Startup synchronization
Random characters	Data order and E pulses
Wrong row	DDRAM offset
Extra old digits	Fixed-width repaint
Repeated actions	Input-event state

A systematic diagnosis

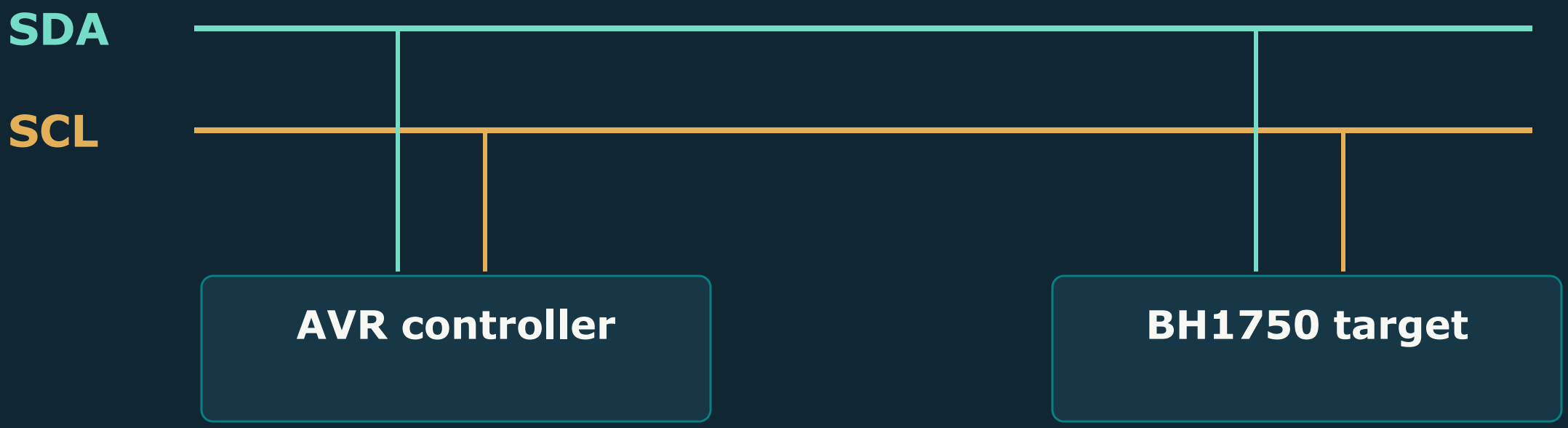
1. Check power, ground and contrast.
2. Verify shield pin mapping and output directions.
3. Observe initialization and E strobes.
4. Verify bytes, RS and DDRAM positioning.
5. Check input events and application state.

LCD and I²C are separate interfaces



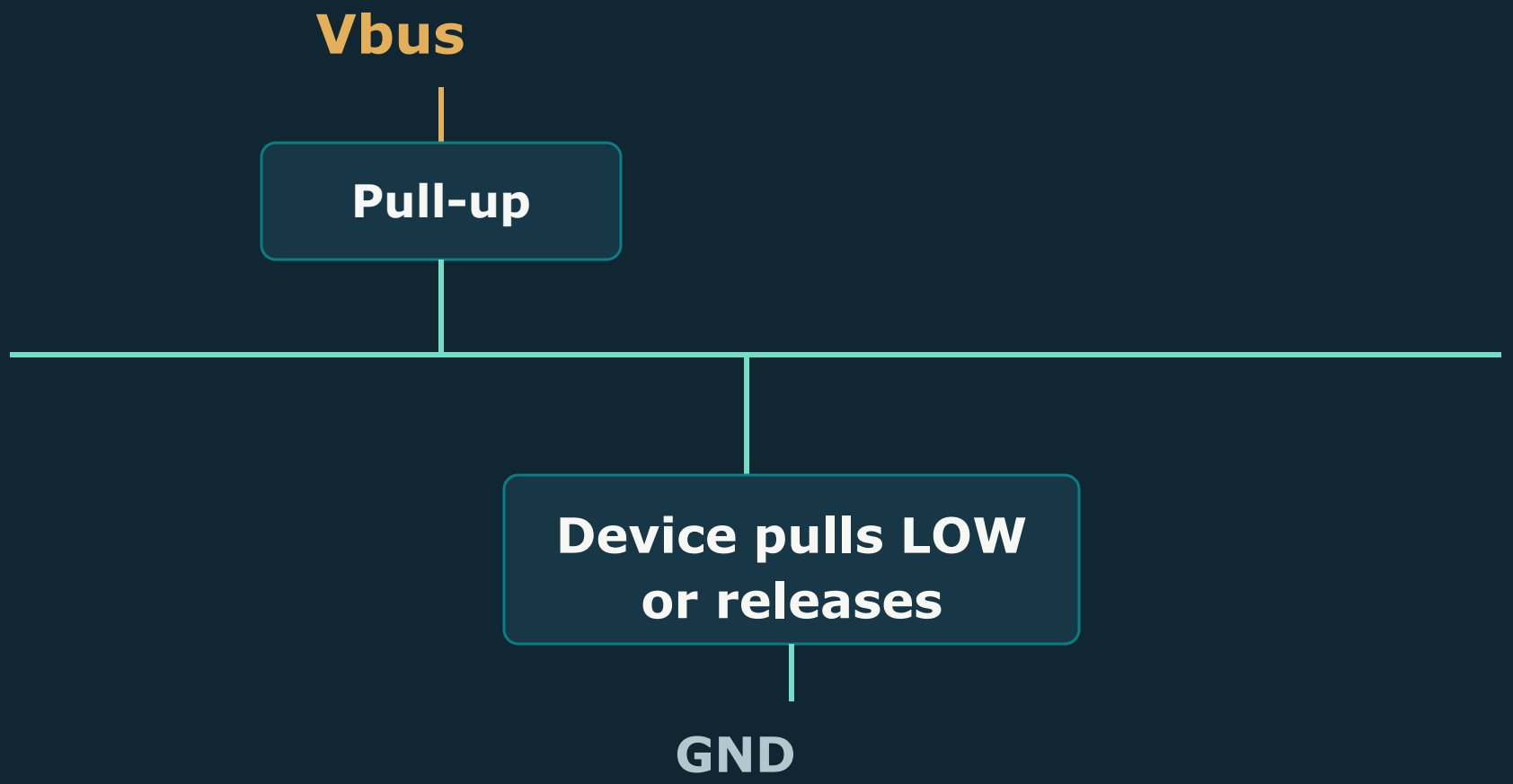
A backpack would add an I²C-to-parallel interface.

A shared bus for addressed devices



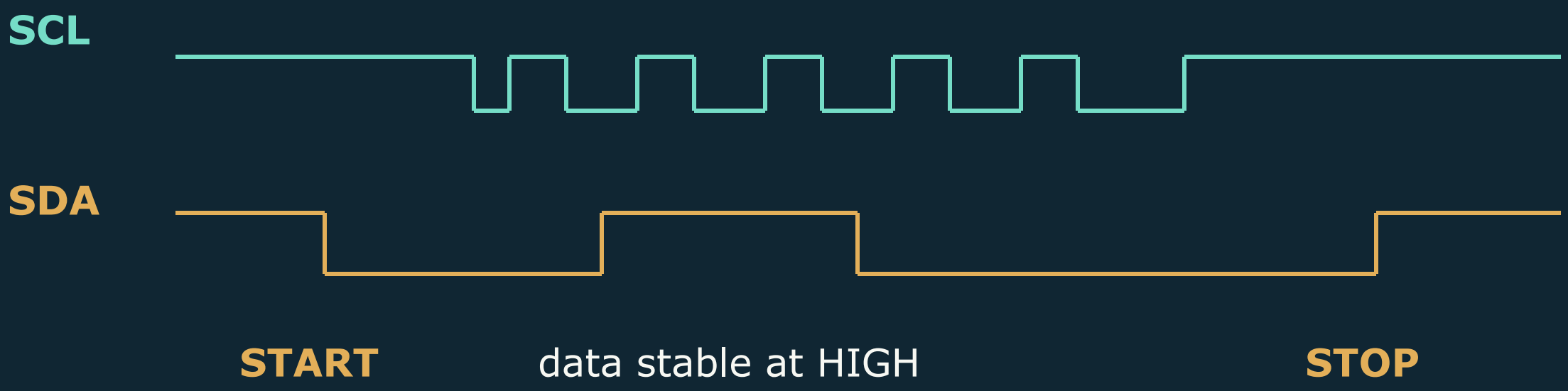
Common ground and compatible signal levels are also required.

Pull-ups and voltage compatibility



HIGH comes from the pull-up.

START, data and STOP



START: SDA falls while SCL is HIGH.
A data bit stays stable while SCL is HIGH.
STOP: SDA rises while SCL is HIGH.

ACK and NACK on the ninth clock

Phase	Receiver	Ninth clock
Address or command write	BH1750	ACK pulls SDA LOW
First measurement byte	AVR	ACK asks for another byte
Final measurement byte	AVR	NACK ends the read

One device, two address bytes

0x23

7-bit address

0x46 write

0x47 read

address byte = (address << 1) | direction

One BH1750 measurement

Write command	Conversion	Read result
01 then 20	Wait ≥180 ms	high, low
Device ACKs	Default MTreg	AVR ACK, then NACK

Each transaction ends with STOP.

A new one-time measurement needs a new command.

Exercise E7: read the laboratory trace

START 46 ACK 20 ACK STOP

Wait ≥180 ms at default MTreg

START 47 ACK 01 ACK 2C NACK STOP

Identify address, command and direction. Who generates the final NACK? Decode the illuminance.

Solution E7: read the laboratory trace

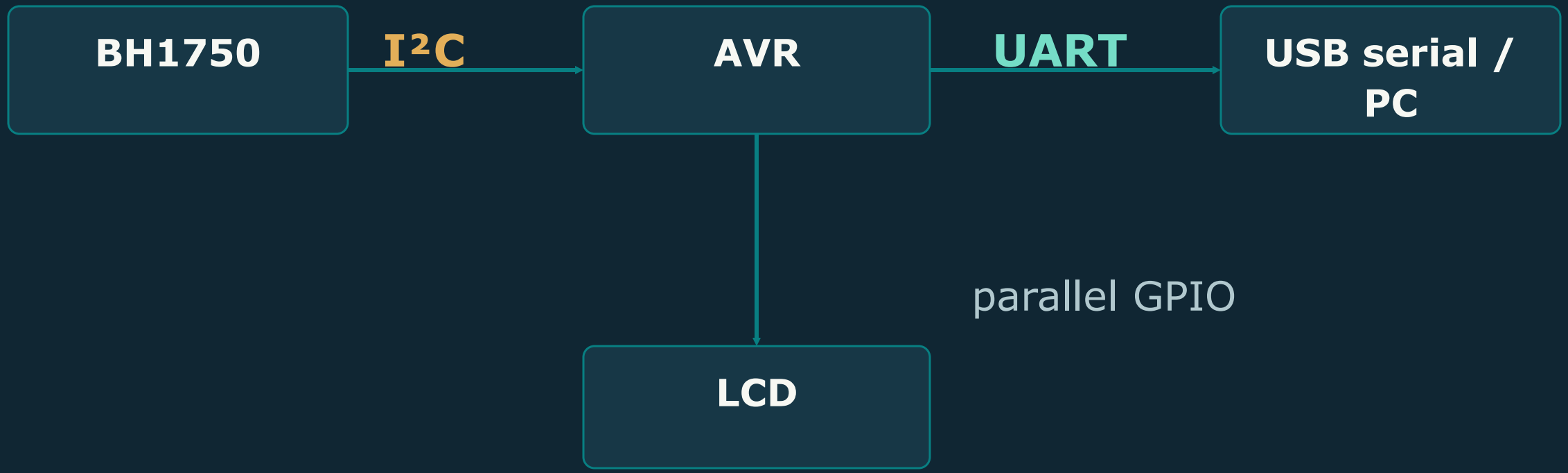
0x46 writes to device 0x23. 0x20 starts a measurement.

After conversion, 0x47 reads the same device.

AVR ACKs 0x01, then NACKs final byte 0x2C.

Raw 300 gives nominal 250 lx.

The measurement and reporting paths



What to remember

The controller has its own state, memory and timing.

RS changes byte meaning. E clocks each nibble.

Four-bit writes send the high nibble first.

Menus need input events and explicit state.

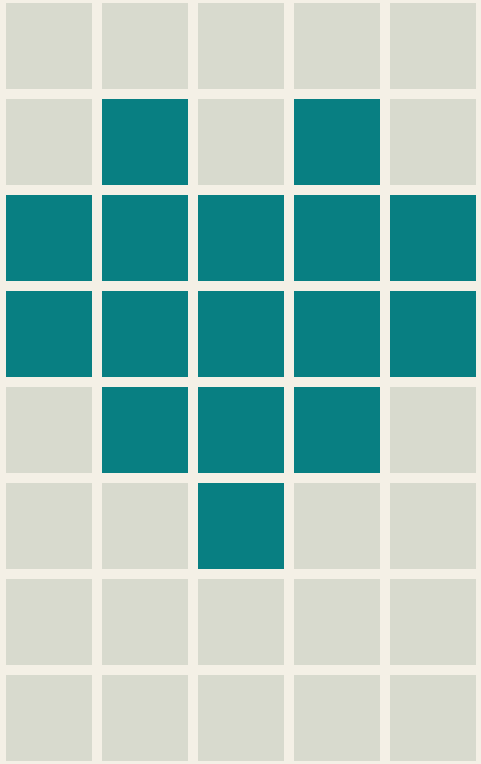
Sensor protocols need addresses, timing and error checks.

Extension: custom characters

CGRAM design and programming

Worked glyphs, zero bytes and diagnosis

A heart as eight five-bit rows



00000	0x00
01010	0x0A
11111	0x1F
11111	0x1F
01110	0x0E
00100	0x04
00000	0x00
00000	0x00

Only the low five bits draw visible dots.

Each byte describes one row.

One glyph occupies eight CGRAM bytes.

Loading and displaying a custom slot

```
lcd_command(0x40 | (2 << 3));
for (uint8_t row = 0; row < 8; ++row)
    lcd_data(pattern[row] & 0x1F);

lcd_cursor(1, 3);
lcd_data(2);
```

CGRAM command base: 0x40

Slot 2 starts at address 16.

After loading, select DDRAM again.

Exercise E9: a smile in slot 1

Rows: 00, 0A, 0A, 00, 11, 0E, 00, 00.

Give the first CGRAM command.

Display the glyph in row 2, column 4.

Explain why lcd_puts("\0") displays nothing.

Solution E9: a smile in slot 1

Slot 1 command = 0x40 | 8 = 0x48.

lcd_glyph(1, smile);

lcd_cursor(1,3); lcd_data(1);

lcd_puts stops at zero. Use lcd_data(0) for slot 0.

Extension: persistent settings

SRAM, Flash, LCD memory and EEPROM

Explicit save, validated load and failure behavior

Where the values live

Memory	Use here	Power loss
AVR Flash	Program instructions	Retains program
AVR SRAM	Working settings and menu state	Loses data
AVR EEPROM	Saved setting record	Retains data
LCD DDRAM/CGRAM	Text codes and custom patterns	Loses data

Explicit save and validated reload



Edit the working copy in SRAM.

Save only on a deliberate user action.

On boot, validate the EEPROM record.

Invalid or incomplete records use defaults.

The eight-byte record

Byte	Meaning
0	Valid marker 0xA6
1	Format version 1
2,3	Threshold, low byte then high byte
4,5	Hysteresis, low byte then high byte
6	Mode: AUTO=0, OFF=1, ON=2
7	XOR checksum seeded with 0x5A

A save interrupted by power loss

1. Invalidate the old record.
2. Write the payload and checksum.
3. Commit the valid marker last.
4. Wait for the final write to finish.

Exercise E10: which value survives

Boot defaults: threshold 300.

Edit threshold to 400. Reset without saving.

Edit to 500, complete SAVE, then power cycle.

The next save stops before the commit marker. Predict each load.

Solution E10: which value survives

Unsaved edit: the valid old record loads, or 300 if none exists.

Completed save: 500 loads if the record validates.

Interrupted save: invalid record gives defaults of 300.

The single-record design does not retain the prior saved value.

EEPROM and running LCD text

Edit a value, save it, and recover it after power-off.

Build a scrolling message with a saved speed setting.

EEPROM remembers saved settings



SRAM holds the working value while the program runs.

EEPROM retains the saved byte when power is removed.

UP/DOWN edit SRAM. SELECT copies the value to EEPROM.

A simple value editor

Value: 12

Saved: 10

UP: add 1

DOWN: subtract 1

SELECT: save

Allowed range: 0–100

Reserve and read one EEPROM byte

```
#include <avr/eeprom.h>
uint8_t EEMEM saved_value;

uint8_t value;
value = eeprom_read_byte(&saved_value);
if (value > 100) value = 10;
```

Save only when SELECT is pressed

```
if (event.press == KEY_SELECT) {
    eeprom_update_byte(&saved_value, value);
    eeprom_busy_wait();
}
```

Verify that saving works

1. Set the value to 12 and press SELECT.

2. Change it to 15 without saving.

3. Reset or power-cycle: the value must return to 12.

4. Save 15, then restart: the value must return to 15.

Shift the display without rewriting text

```
lcd_command(0x18);  // Shift display LEFT
lcd_command(0x1C);  // Shift display RIGHT
```

One command moves one character position.

Repeated commands create motion.

DDRAM character bytes stay unchanged.

One left shift changes the visible window

MICROCONTROLLER

ICROCONTROLLER

Top: before shift

Bottom: after 0x18

These show two snapshots of the same row.

The display has hidden positions

Each internal row has 40 character positions.

Only 16 positions per row are visible at once.

Both rows move together during a display shift.

Fill hidden cells with spaces; repeated shifts wrap after 40 steps.

Assignment: scrolling message

Create a repeating message on row 1 of the shield.

At startup, load the saved speed and move left.

LEFT selects continuous leftward motion.

RIGHT selects continuous rightward motion.

Assignment: speed setting

SELECT opens the speed setting and pauses motion.

UP increases speed; DOWN decreases speed.

Keep speed within 1–10 and show its current value.

SELECT saves speed to EEPROM and returns to the running message.

A higher speed needs a shorter interval

Speed	Time per shift	Shifts per second
1	1000 ms	1
5	600 ms	about 1.67
10	100 ms	10

Assignment: implementation requirements

Use lcd_command(0x18) and lcd_command(0x1C).

Reuse lcd.c, lcd.h, keys.c and keys.h unchanged.

Poll keys every 10 ms, even between shift steps.

Use separate RUN and SPEED states; save only on SELECT.

Assignment: acceptance tests

Reverse direction while the message is moving.

At speed 1 and 10, confirm roughly 1 s and 0.1 s per step.

Open settings: the menu must remain stationary.

Save speed 8, restart, and confirm speed 8 is restored.

Demonstration solution: two states



RUN: read LEFT/RIGHT; shift when the interval expires.

SPEED: read UP/DOWN; keep the menu stationary.

SELECT changes state; leaving SPEED also saves.

Diagnose these three mistakes

The menu also moves: the display shift affects both rows.

UP makes movement slower: the delay increased with speed.

Every restart returns to default: saving or EEPROM retention failed.